

Venture Brief → Technical Spec

Paste a founder brief. Get a production-aware technical specification.

Founder Brief

So basically the problem we're trying to solve is, like, restaurant staff scheduling is way more chaotic than people outside the industry realize. Especially for small and mid-sized restaurants, cafes, bars, quick-service places, that kind of thing. It's not like big enterprise workforce management where they have some HR department and proper tools and all that. Most of these owners or outlet managers are literally running everything through WhatsApp groups, Excel

Generate Spec

Spec generated.

Domain & Context

DOMAIN: Restaurant staff scheduling and shift management

PRIMARY USERS:

- Restaurant managers or owners (primary user)
- Restaurant staff (servers, kitchen assistants, etc.)

CORE ACTIONS:

1. Managers create and publish weekly schedules with shifts.
2. Staff view their shifts, claim open shifts, and request shift swaps.
3. Managers approve shift swaps and manage shift assignments.
4. Staff receive reminders for upcoming shifts and confirm their shifts.

Production Flags

SCALE RISKS:

1. **Database Performance:** With an increase in users (e.g., 10x), the database may struggle with concurrent read/write operations, leading to slow response times or timeouts.
2. **API Rate Limiting:** Increased API calls from multiple users could exceed rate limits, causing delays or failures in shift claims and updates.
3. **Notification Overload:** A surge in notifications (e.g., reminders, approvals) could overwhelm the notification service, leading to missed alerts or delays.
4. **Real-time Updates:** If implemented, real-time updates may become inefficient with high user

- Managers monitor shift coverage and address open shifts or pending requests.

KEY CONSTRAINTS:

- Must be mobile-first; desktop usage is secondary.
- Simple and lightweight interface; avoid complex HR software features.
- Integration with WhatsApp is a future consideration, not part of MVP.
- Support for multiple restaurant locations and staff roles.
- Must handle staff availability and eligibility for shifts.
- Notifications and reminders need to be effective and timely.

WHAT IS BEING REPLACED: The current chaotic scheduling process managed through WhatsApp groups, Excel sheets, and informal communication methods.

Data Model

Database Tables for MVP

- restaurants**
 - id** (UUID, Primary Key)
Stores the unique identifier for each restaurant.
 - name** (VARCHAR)
Stores the name of the restaurant.
 - created_at** (TIMESTAMP)
Stores the timestamp when the restaurant was created.
 - updated_at** (TIMESTAMP)
Stores the timestamp when the restaurant was last updated.

counts, causing lag or failures in displaying current shift statuses.

- User Interface Clutter:** A larger user base may lead to more complexity in the UI, making it harder for staff to navigate and find relevant information quickly.

SECURITY CONCERNS:

- Authentication:** Ensure secure authentication methods (e.g., OTP) to prevent unauthorized access to manager and staff accounts.
- Data Privacy:** Protect sensitive user data (e.g., phone numbers, shift assignments) with encryption both in transit and at rest.
- Role-based Access Control:** Implement strict permissions to ensure users only access data relevant to their roles (e.g., managers vs. staff).
- Audit Logging:** Maintain logs of all significant actions (e.g., shift claims, swaps) to track changes and resolve disputes.
- Data Breaches:** Regularly assess vulnerabilities and ensure compliance with data protection regulations (e.g., GDPR, CCPA).

DEPLOYMENT CONSIDERATIONS:

- Hosting Environment:** Choose a scalable cloud provider (e.g., AWS, GCP) to handle varying loads and ensure high availability.
- Database Management:** Use a relational database (e.g., PostgreSQL) with proper indexing and optimization for performance.
- Load Balancing:** Implement load balancers to distribute traffic evenly across servers, preventing overload on any single instance.
- Monitoring and Logging:** Set up monitoring tools (e.g., New Relic, Datadog) to track

2. locations

- **id** (UUID, Primary Key)
Stores the unique identifier for each location.
- **restaurant_id** (UUID, Foreign Key)
References the restaurant this location belongs to.
- **name** (VARCHAR)
Stores the name of the location (e.g., Outlet A).
- **timezone** (VARCHAR)
Stores the timezone for scheduling shifts at this location.
- **created_at** (TIMESTAMP)
Stores the timestamp when the location was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the location was last updated.

3. users

- **id** (UUID, Primary Key)
Stores the unique identifier for each user (staff or manager).
- **restaurant_id** (UUID, Foreign Key)
References the restaurant this user belongs to.
- **name** (VARCHAR)
Stores the name of the user.
- **phone_number** (VARCHAR)
Stores the phone number for user authentication and notifications.
- **role** (VARCHAR)
Stores the role of the user (e.g., server, manager).
- **created_at** (TIMESTAMP)
Stores the timestamp when the user was created.

application performance and error rates, along with logging for troubleshooting.

5. **Backup and Recovery:** Establish regular backup procedures and a disaster recovery plan to prevent data loss.

WHAT TO DEFER:

1. **Payroll Integration:** Integrating with payroll systems can be complex and is not needed for MVP; focus on scheduling first.
2. **Advanced Availability Features:** Recurring availability and detailed availability management can be added later; start with basic unavailable date marking.
3. **WhatsApp Integration:** While valuable, this can be deferred to a later phase to focus on core scheduling functionalities.
4. **AI Scheduling Suggestions:** AI-driven recommendations for staffing can be considered in future iterations but are not necessary for the initial launch.
5. **Complex Reporting and Analytics:** Detailed analytics and reporting features can be added later; the initial focus should be on operational functionality.

Open Questions

1. **What specific features must be included in the MVP, and which can be deferred to later versions?**
 - **Why it blocks development:** Without a clear definition of MVP features, the development team may waste time building unnecessary functionalities or miss critical features that are essential for the product's success.
 - **Who should answer it:** Founder.

- **updated_at** (TIMESTAMP)
Stores the timestamp when the user was last updated.

4. shifts

- **id** (UUID, Primary Key)
Stores the unique identifier for each shift.
- **location_id** (UUID, Foreign Key)
References the location where the shift is scheduled.
- **date** (DATE)
Stores the date of the shift.
- **start_time** (TIME)
Stores the start time of the shift.
- **end_time** (TIME)
Stores the end time of the shift.
- **role** (VARCHAR)
Stores the role required for the shift (e.g., server, cook).
- **number_of_people** (INTEGER)
Stores the number of staff needed for the shift.
- **status** (VARCHAR)
Stores the status of the shift (e.g., draft, published, open, filled).
- **created_at** (TIMESTAMP)
Stores the timestamp when the shift was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the shift was last updated.

5. shift_assignments

- **id** (UUID, Primary Key)
Stores the unique identifier for each shift assignment.
- **shift_id** (UUID, Foreign Key)
References the shift being assigned.
- **user_id** (UUID, Foreign Key)
References the user assigned to the shift.

2. What is the preferred authentication method for staff accounts (e.g., OTP, email/password, invite link)?

- **Why it blocks development:** The choice of authentication method affects the user onboarding process and overall user experience. If not defined, it could lead to a cumbersome login process that deters staff from using the app.
- **Who should answer it:** Founder or designer.

3. How should shift claiming and assignment approval work (e.g., first-come-first-served vs. manager approval)?

- **Why it blocks development:** This decision impacts the core functionality of the scheduling system and affects how staff interact with the app. Ambiguity here could lead to confusion in the user experience.
- **Who should answer it:** Founder.

4. What is the expected process for notifying staff about schedule changes, and how should confirmations be handled?

- **Why it blocks development:** Without a clear notification and confirmation process, the system may fail to keep staff informed about important changes, leading to operational chaos similar to the current WhatsApp method.
- **Who should answer it:** Founder or designer.

5. What are the specific roles and permissions that need to be defined for staff and managers?

- **Why it blocks development:** Role definitions are crucial for implementing access control and ensuring users can only

- **status** (VARCHAR)
Stores the status of the assignment (e.g., assigned, confirmed, pending confirmation).
- **created_at** (TIMESTAMP)
Stores the timestamp when the assignment was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the assignment was last updated.

6. swap_requests

- **id** (UUID, Primary Key)
Stores the unique identifier for each swap request.
- **shift_assignment_id** (UUID, Foreign Key)
References the shift assignment being swapped.
- **requesting_user_id** (UUID, Foreign Key)
References the user requesting the swap.
- **target_user_id** (UUID, Foreign Key)
References the user with whom the swap is requested.
- **status** (VARCHAR)
Stores the status of the swap request (e.g., pending, approved, rejected).
- **created_at** (TIMESTAMP)
Stores the timestamp when the swap request was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the swap request was last updated.

7. notifications

- **id** (UUID, Primary Key)
Stores the unique identifier for each notification.
- **user_id** (UUID, Foreign Key)
References the user to whom the notification is sent.

perform actions relevant to their roles.
Lack of clarity here could lead to security vulnerabilities.

- **Who should answer it:** Founder.

6. What should the user interface look like for both managers and staff, and what are the key screens?

- **Why it blocks development:** UI design directly influences user experience. If the design is not finalized, developers may create a product that does not meet user needs or expectations.

- **Who should answer it:** Designer.

7. What are the notification preferences for reminders (e.g., timing, method of delivery)?

- **Why it blocks development:** Notification preferences will dictate how the backend handles reminders and alerts. If not defined, it could lead to ineffective communication with staff.

- **Who should answer it:** Founder.

8. What are the specific edge cases that need to be handled for shift swaps and claims?

- **Why it blocks development:** Identifying edge cases is essential for ensuring the system behaves correctly under various scenarios. Without this, the system may have bugs that disrupt operations.
- **Who should answer it:** Founder or designer.

9. What is the plan for handling staff availability and unavailability in the MVP?

- **Why it blocks development:** The approach to managing availability affects how shifts are assigned and claimed. If not clarified, it could lead to scheduling conflicts and staff dissatisfaction.

- **message** (TEXT)
Stores the content of the notification.
- **sent_at** (TIMESTAMP)
Stores the timestamp when the notification was sent.
- **type** (VARCHAR)
Stores the type of notification (e.g., reminder, confirmation, alert).

8. availability

- **id** (UUID, Primary Key)
Stores the unique identifier for each availability entry.
- **user_id** (UUID, Foreign Key)
References the user indicating their availability.
- **date** (DATE)
Stores the date for which the availability is indicated.
- **is_available** (BOOLEAN)
Indicates whether the user is available on that date.
- **created_at** (TIMESTAMP)
Stores the timestamp when the availability was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the availability was last updated.

Summary

This database schema supports the core functionalities required for the MVP, including managing restaurants, locations, users, shifts, assignments, swap requests, notifications, and availability, without over-engineering or including unnecessary complexity.

API Design

- **Who should answer it:** Founder.

10. What are the long-term goals for payroll integration, and how should the current data model accommodate future needs?

- **Why it blocks development:**
Understanding future requirements for payroll integration will inform the initial data model design, ensuring it can support future functionalities without requiring significant rework.
- **Who should answer it:** Founder.

Week 1 Task Breakdown

MUST BUILD (Week 1 Tasks)

1. Set up the development environment

- Establishing a consistent development environment is crucial for all team members to ensure smooth collaboration and code integration.

2. Create the database schema

- Implementing the database schema is foundational as it defines how data will be structured and stored, enabling subsequent development of API endpoints.

3. Implement the REST API for Restaurants

- Building the API for managing restaurants is essential as it serves as the entry point for creating and managing restaurant data, which is necessary for all other functionalities.

4. Implement the REST API for Users

- Developing the user management API is critical because it allows for the creation

REST API Design

Restaurants

1. POST /restaurants

- Creates a new restaurant.
- Request Body:**

```
{  
  "name": "string"  
}
```

- Response:**

```
{  
  "id": "UUID",  
  "name": "string",  
  "created_at": "TIMESTAMP",  
  "updated_at": "TIMESTAMP"  
}
```

2. GET /restaurants

- Retrieves a list of all restaurants.
- Response:**

```
[  
  {  
    "id": "UUID",  
    "name": "string",  
    "created_at": "TIMESTAMP",  
    "updated_at": "TIMESTAMP"  
  }  
]
```

Locations

1. POST /locations

- Creates a new location for a restaurant.
- Request Body:**

and authentication of restaurant staff and managers, enabling user-specific functionalities.

5. Create basic front-end structure (mobile-first)

- Establishing the front-end structure is important to ensure that the application is mobile-friendly from the start, aligning with the project's mobile-first approach.

DEFER TO WEEK 2

1. Implement the REST API for Locations

- While necessary, this can wait until the basic user and restaurant functionalities are in place, as locations build on those foundations.

2. Implement the REST API for Shifts

- Shift management is important but can be deferred until the user and restaurant APIs are functional, as shifts will depend on those entities.

3. Set up initial user authentication

- User authentication is important but can be implemented after the basic user management API, as it will build on the existing user functionalities.

DO NOT TOUCH YET

1. Integration with WhatsApp

- This is a future consideration and should not be addressed until the core functionalities are fully developed and tested.

2. Advanced features like shift swaps and notifications

```
{
  "restaurant_id": "UUID",
  "name": "string",
  "timezone": "string"
}
```

- **Response:**

```
{
  "id": "UUID",
  "restaurant_id": "UUID",
  "name": "string",
  "timezone": "string",
  "created_at": "TIMESTAMP",
  "updated_at": "TIMESTAMP"
}
```

2. GET /locations

- Retrieves a list of all locations.
- **Response:**

```
[
  {
    "id": "UUID",
    "restaurant_id": "UUID",
    "name": "string",
    "timezone": "string",
    "created_at": "TIMESTAMP",
    "updated_at": "TIMESTAMP"
  }
]
```

Users

1. POST /users

- Creates a new user for a restaurant.
- **Request Body:**

```
{
  "restaurant_id": "UUID",
  "name": "string",
  "phone_number": "string",
}
```

- These features can be complex and are not essential for the MVP; focus should remain on core functionalities first.

3. Detailed reporting and analytics

- Reporting features can be added later, as they are not critical to the initial user experience and can be built once the core scheduling features are stable.


```
"role": "string"
}
```

- **Response:**

```
{
  "id": "UUID",
  "restaurant_id": "UUID",
  "name": "string",
  "phone_number": "string",
  "role": "string",
  "created_at": "TIMESTAMP",
  "updated_at": "TIMESTAMP"
}
```

2. GET /users

- Retrieves a list of all users.
- **Response:**

```
[
  {
    "id": "UUID",
    "restaurant_id": "UUID",
    "name": "string",
    "phone_number": "string",
    "role": "string",
    "created_at": "TIMESTAMP",
    "updated_at": "TIMESTAMP"
  }
]
```

Shifts

1. POST /shifts

- Creates a new shift for a location.
- **Request Body:**

```
{
  "location_id": "UUID",
  "date": "DATE",
  "start_time": "TIME",
  "end_time": "TIME",
  "role": "string",
}
```

```
"number_of_people": "integer",  
"status": "string"  
}
```

- **Response:**

```
{  
  "id": "UUID",  
  "location_id": "UUID",  
  "date": "DATE",  
  "start_time": "TIME",  
  "end_time": "TIME",  
  "role": "string",  
  "number_of_people": "integer",  
  "status": "string",  
  "created_at": "TIMESTAMP",  
  "updated_at": "TIMESTAMP"  
}
```

2. GET /shifts

- Retrieves a list of all shifts.

- **Response:**

```
[  
  {  
    "id": "UUID",  
    "location_id": "UUID",  
    "date": "DATE",  
    "start_time": "TIME",  
    "end_time": "TIME",  
    "role": "string",  
    "number_of_people": "integer",  
    "status": "string",  
    "created_at": "TIMESTAMP",  
    "updated_at": "TIMESTAMP"  
  }  
]
```

Shift Assignments

1. POST /shift_assignments

- Assigns a user to a shift.

- **Request Body:**

```
{
  "shift_id": "UUID",
  "user_id": "UUID",
  "status": "string"
}
```

- **Response:**

```
{
  "id": "UUID",
  "shift_id": "UUID",
  "user_id": "UUID",
  "status": "string",
  "created_at": "TIMESTAMP",
  "updated_at": "TIMESTAMP"
}
```

2. GET /shift_assignments

- Retrieves a list of all shift assignments.

- **Response:**

```
[
  {
    "id": "UUID",
    "shift_id": "UUID",
    "user_id": "UUID",
    "status": "string",
    "created_at": "TIMESTAMP",
    "updated_at": "TIMESTAMP"
  }
]
```

Swap Requests

1. POST /swap_requests

- Creates a new swap request for a shift assignment.

- **Request Body:**

```
{
  "shift_assignment_id": "UUID",

```

```
"requesting_user_id": "UUID",  
"target_user_id": "UUID",  
"status": "string"  
}
```

- **Response:**

```
{  
  "id": "UUID",  
  "shift_assignment_id": "UUID",  
  "requesting_user_id": "UUID",  
  "target_user_id": "UUID",  
  "status": "string",  
  "created_at": "TIMESTAMP",  
  "updated_at": "TIMESTAMP"  
}
```

2. GET /swap_requests

- Retrieves a list of all swap requests.

- **Response:**

```
[  
  {  
    "id": "UUID",  
    "shift_assignment_id": "UUID",  
    "requesting_user_id": "UUID",  
    "target_user_id": "UUID",  
    "status": "string",  
    "created_at": "TIMESTAMP",  
    "updated_at": "TIMESTAMP"  
  }  
]
```

Notifications

1. POST /notifications

- Sends a notification to a user.
- **Request Body:**

```
{  
  "user_id": "UUID",  
  "message": "string",  
}
```

```
"type": "string"
}
```

- **Response:**

```
{
  "id": "UUID",
  "user_id": "UUID",
  "message": "string",
  "sent_at": "TIMESTAMP",
  "type": "string"
}
```

2. GET /notifications

- Retrieves a list of notifications for all users.
- **Response:**

```
[
  {
    "id": "UUID",
    "user_id": "UUID",
    "message": "string",
    "sent_at": "TIMESTAMP",
    "type": "string"
  }
]
```

Availability

1. POST /availability

- Submits availability for a user.
- **Request Body:**

```
{
  "user_id": "UUID",
  "date": "DATE",
  "is_available": "boolean"
}
```

- **Response:**

```
{
  "id": "UUID",
  "user_id": "UUID",
  "date": "DATE",
  "is_available": "boolean",
  "created_at": "TIMESTAMP",
  "updated_at": "TIMESTAMP"
}
```

2. GET /availability

- Retrieves a list of availability entries for all users.
- Response:**

```
[
```

View Prompts Used

Stage 1 — Extract

```
"user_id": "UUID",
"date": "DATE",
```

Read this founder brief [and](#) extract the following. Be specific.

BRIEF:

So basically the problem we're trying to solve [is](#), like, restaurant staff scheduling. The current process [is](#) honestly kind of broken. A manager will usually make the schedule. And [for](#) restaurant owners, this [is](#) super painful because one missing person can mess up the whole week. So what we want to build [is](#) basically a mobile-first scheduling [and](#) shift management tool. The first user [is](#) probably the restaurant manager [or](#) owner. They need to create shifts. Actually, we probably need both. Some shifts are assigned directly because the manager wants them. For staff, they should be able to log [in and](#) see "My Shifts" [for](#) the week. [V](#) Mobile-first [is](#) very important. I don't think staff will use desktop. Management tool. The user size per restaurant [is](#) probably around 50 to 200 staff, but smaller restaurants might have less. Roles are also important. You can't just let anyone claim any shift. A kitchen manager can assign shifts. Availability [is](#) another thing. Staff should maybe be able to submit availability

The biggest pain point **is** shift swaps. Right now someone says “Can anyone co-own a shift?”

Actually overtime **and** labor rules might matter later, but maybe **not** now. Pay is a pain point.

For reminders, we need automatic notifications. Staff should get reminded before their shift.

Managers need a view of coverage. Like **for** each day, are all shifts filled? Can we see who is on which shift?

The scheduling UI should probably have a weekly calendar view. But **for** mobile, a list view might be better.

Authentication-wise, we need manager accounts **and** staff accounts. Staff onboarding should be simple.

We should support permissions. Owner/admin can manage billing, locations, staff, etc.

Payroll integration **is** later, but we should know what it means. Could integrate with a payroll provider.

Important edge cases: staff claiming two overlapping shifts should **not** be allowed.

Maybe there’s also a need **for** recurring shifts. Many restaurants have repeat shifts.

Another thing **is** staff availability **and** leave. If someone **is** on leave, they shouldn’t be assigned shifts.

I’m **not** sure about pricing but eventually this probably charges per restaurant or per shift.

The main product objects I imagine are restaurants, locations, users, roles, shifts, and coverage.

The tone of the product should be very operational **and** practical. Restaurant scheduling is a pain.

We may eventually add AI suggestions, like “based on sales forecast, you need more staff on Friday night.”

For the staff experience, it should be dead simple. They open it **and** see: next shift, swap, etc.

For the manager, the most important screens are probably:

1. weekly schedule,
2. open shift coverage,
3. swap approvals,
4. staff list,
5. notification/reminder settings.

I’m also thinking about audit logs because disputes happen. Like someone say “I didn’t swap that shift!”

We need statuses. Shift could be draft, published, open, filled, completed, etc.

One thing I worry about **is** managers making schedule changes after publishing.

Also, we need to think about bulk actions. Managers don't want to create one

For MVP, maybe the simplest flow is:

Manager creates restaurant and location.

Manager adds staff with names, phone numbers, roles.

Manager creates shifts for the week, either assigned or open.

Manager publishes schedule.

Staff receive invite/reminder and can see shifts.

Staff confirm shifts.

Staff request swaps or claim open shifts.

Manager approves swaps.

System sends reminders before shifts.

That's probably the core.

We don't need payroll in MVP, but maybe we include export scheduled hours. S

Technically, the API should be built in a way that supports mobile clients.

Also, staff count of 50-200 means it's not huge scale, but we should support

I don't know how much detail we need in the prompt output, but the chain sho

- * product summary,
- * user roles,
- * core entities,
- * workflows,
- * API endpoints,
- * MVP scope,
- * future scope,
- * constraints,
- * edge cases,
- * notification requirements,
- * data model,
- * permissions.

And it should separate what is confirmed from what is optional or future. Fo

The biggest thing is: don't overbuild. The founder probably wants everything

Also please note that this is for small restaurant owners, not enterprise. S

Output exactly this structure:

DOMAIN: (what industry/space is this)

PRIMARY USERS: (who uses this, be specific)

CORE ACTIONS: (what are the 3-5 main things users do)

KEY CONSTRAINTS: (technical or business constraints mentioned)

WHAT IS BEING REPLACED: (what existing solution/workflow does this replace)
Stage 2 — Data Model

You are designing a database for this product.

ORIGINAL BRIEF:

So basically the problem we're trying to solve is, like, restaurant staff scheduling. The current process is honestly kind of broken. A manager will usually make shifts for restaurant owners, this is super painful because one missing person can mess up the whole schedule. So what we want to build is basically a mobile-first scheduling and shift management system. The first user is probably the restaurant manager or owner. They need to create shifts for staff. Actually, we probably need both. Some shifts are assigned directly because the manager knows who needs them. For staff, they should be able to log in and see "My Shifts" for the week. Mobile-first is very important. I don't think staff will use desktop. Manager and staff both. The user size per restaurant is probably around 50 to 200 staff, but smaller for some. Roles are also important. You can't just let anyone claim any shift. A kitchen helper can't be a server. Availability is another thing. Staff should maybe be able to submit availability. The biggest pain point is shift swaps. Right now someone says "Can anyone cover this shift?" and it's a mess. Actually overtime and labor rules might matter later, but maybe not now. Payroll integration is later, but we should know what it means. Could integrate with existing payroll systems. For reminders, we need automatic notifications. Staff should get reminded before shifts. Managers need a view of coverage. Like for each day, are all shifts filled? The scheduling UI should probably have a weekly calendar view. But for mobile, a list view might be better. Authentication-wise, we need manager accounts and staff accounts. Staff onboarding should be simple. We should support permissions. Owner/admin can manage billing, locations, staff. Payroll integration is later, but we should know what it means. Could integrate with existing payroll systems.

Important edge cases: staff claiming two overlapping shifts should **not** be allowed.

Maybe there's also a need **for** recurring shifts. Many restaurants have repeat shifts.

Another thing **is** staff availability **and** leave. If someone **is** on leave, they can't work.

I'm **not** sure about pricing but eventually this probably charges per restaurant per shift.

The main product objects I imagine are restaurants, locations, users, roles, permissions.

The tone of the product should be very operational **and** practical. Restaurant management is a practical thing.

We may eventually add AI suggestions, like "based on sales forecast, you need more staff on Friday night."

For the staff experience, it should be dead simple. They open it **and** see: next shifts.

For the manager, the most important screens are probably:

1. weekly schedule,
2. open shift coverage,
3. swap approvals,
4. staff list,
5. notification/reminder settings.

I'm also thinking about audit logs because disputes happen. Like someone say "I didn't work that shift!"

We need statuses. Shift could be draft, published, open, filled, completed, closed.

One thing I worry about **is** managers making schedule changes after publishing.

Also, we need to think about bulk actions. Managers don't want to create one shift at a time.

For MVP, maybe the simplest flow **is**:

- Manager creates restaurant **and** location.
- Manager adds staff **with** names, phone numbers, roles.
- Manager creates shifts **for** the week, either assigned **or** open.
- Manager publishes schedule.
- Staff receive invite/reminder **and** can see shifts.
- Staff confirm shifts.
- Staff request swaps **or** claim open shifts.
- Manager approves swaps.
- System sends reminders before shifts.

That's probably the core.

We don't need payroll **in** MVP, but maybe we include export scheduled hours. S

Technically, the API should be built **in** a way that supports mobile clients.

Also, staff count of **50-200** means it's **not** huge scale, but we should support

I don't know how much detail we need **in** the prompt output, but the chain sho

- ★ product summary,
- ★ user roles,
- ★ core entities,
- ★ workflows,
- ★ API endpoints,
- ★ MVP scope,
- ★ future scope,
- ★ constraints,
- ★ edge cases,
- ★ notification requirements,
- ★ data model,
- ★ permissions.

And it should separate what **is** confirmed **from** what **is** optional **or** future. Fo

The biggest thing **is**: don't overbuild. The founder probably wants everything

Also please note that this **is for** small restaurant owners, **not** enterprise. S

EXTRACTED CONTEXT:

DOMAIN: Restaurant staff scheduling **and** shift management

PRIMARY USERS:

- Restaurant managers **or** owners (primary user)
- Restaurant staff (servers, kitchen assistants, etc.)

CORE ACTIONS:

1. Managers create **and** publish weekly schedules **with** shifts.
2. Staff view their shifts, claim open shifts, **and** request shift swaps.
3. Managers approve shift swaps **and** manage shift assignments.
4. Staff receive reminders **for** upcoming shifts **and** confirm their shifts.
5. Managers monitor shift coverage **and** address open shifts **or** pending reques

KEY CONSTRAINTS:

- Must be mobile-first; desktop usage **is** secondary.
- Simple **and** lightweight interface; avoid complex HR software features.
- Integration **with** WhatsApp **is** a future consideration, **not** part of MVP.

- Support **for** multiple restaurant locations **and** staff roles.
- Must handle staff availability **and** eligibility **for** shifts.
- Notifications **and** reminders need to be effective **and** timely.

WHAT IS BEING REPLACED:

The current chaotic scheduling process managed through WhatsApp groups, Excel

List every database table needed. For each table:

- Table name
- Fields (name **and** type)
- One line explaining what it stores

Stage 3 — API Design Only include tables the MVP actually needs. No over-engineering.

Design the REST API **for** this product.

DATA MODEL:

Database Tables for MVP

1. **restaurants**

- **id** (UUID, Primary Key)
Stores the unique identifier **for** each restaurant.
- **name** (VARCHAR)
Stores the name of the restaurant.
- **created_at** (TIMESTAMP)
Stores the timestamp when the restaurant was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the restaurant was last updated.

2. **locations**

- **id** (UUID, Primary Key)
Stores the unique identifier **for** each location.
- **restaurant_id** (UUID, Foreign Key)
References the restaurant this location belongs to.
- **name** (VARCHAR)
Stores the name of the location (e.g., Outlet A).
- **timezone** (VARCHAR)
Stores the timezone **for** scheduling shifts at this location.
- **created_at** (TIMESTAMP)
Stores the timestamp when the location was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the location was last updated.

3. **users**

- **id** (UUID, Primary Key)
Stores the unique identifier **for** each user (staff **or** manager).

- **restaurant_id** (UUID, Foreign Key)
References the restaurant this user belongs to.
- **name** (VARCHAR)
Stores the name of the user.
- **phone_number** (VARCHAR)
Stores the phone number for user authentication and notifications.
- **role** (VARCHAR)
Stores the role of the user (e.g., server, manager).
- **created_at** (TIMESTAMP)
Stores the timestamp when the user was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the user was last updated.

4. **shifts**

- **id** (UUID, Primary Key)
Stores the unique identifier for each shift.
- **location_id** (UUID, Foreign Key)
References the location where the shift is scheduled.
- **date** (DATE)
Stores the date of the shift.
- **start_time** (TIME)
Stores the start time of the shift.
- **end_time** (TIME)
Stores the end time of the shift.
- **role** (VARCHAR)
Stores the role required for the shift (e.g., server, cook).
- **number_of_people** (INTEGER)
Stores the number of staff needed for the shift.
- **status** (VARCHAR)
Stores the status of the shift (e.g., draft, published, open, filled).
- **created_at** (TIMESTAMP)
Stores the timestamp when the shift was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the shift was last updated.

5. **shift_assignments**

- **id** (UUID, Primary Key)
Stores the unique identifier for each shift assignment.
- **shift_id** (UUID, Foreign Key)
References the shift being assigned.
- **user_id** (UUID, Foreign Key)
References the user assigned to the shift.
- **status** (VARCHAR)
Stores the status of the assignment (e.g., assigned, confirmed, pending).
- **created_at** (TIMESTAMP)
Stores the timestamp when the assignment was created.

- **updated_at** (TIMESTAMP)
Stores the timestamp when the assignment was last updated.

6. **swap_requests**

- **id** (UUID, Primary Key)
Stores the unique identifier for each swap request.
- **shift_assignment_id** (UUID, Foreign Key)
References the shift assignment being swapped.
- **requesting_user_id** (UUID, Foreign Key)
References the user requesting the swap.
- **target_user_id** (UUID, Foreign Key)
References the user with whom the swap is requested.
- **status** (VARCHAR)
Stores the status of the swap request (e.g., pending, approved, rejected).
- **created_at** (TIMESTAMP)
Stores the timestamp when the swap request was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the swap request was last updated.

7. **notifications**

- **id** (UUID, Primary Key)
Stores the unique identifier for each notification.
- **user_id** (UUID, Foreign Key)
References the user to whom the notification is sent.
- **message** (TEXT)
Stores the content of the notification.
- **sent_at** (TIMESTAMP)
Stores the timestamp when the notification was sent.
- **type** (VARCHAR)
Stores the type of notification (e.g., reminder, confirmation, alert).

8. **availability**

- **id** (UUID, Primary Key)
Stores the unique identifier for each availability entry.
- **user_id** (UUID, Foreign Key)
References the user indicating their availability.
- **date** (DATE)
Stores the date for which the availability is indicated.
- **is_available** (BOOLEAN)
Indicates whether the user is available on that date.
- **created_at** (TIMESTAMP)
Stores the timestamp when the availability was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the availability was last updated.

Summary

This database schema supports the core functionalities required **for** the MVP,

For each endpoint list:

- Method **+** path (e.g. POST **/**shifts)
- What it does **in** one sentence
- Request body fields (**if** any)
- Response shape

Only include endpoints needed **for** the core actions. Group by feature.

Stage 4 — Production Flags

You are a senior engineer reviewing this project before it goes to production.

BRIEF:

So basically the problem we're trying to solve **is**, like, restaurant staff scheduling.

The current process **is** honestly kind of broken. A manager will usually make shifts for staff.

And **for** restaurant owners, this **is** super painful because one missing person can mess up the whole schedule.

So what we want to build **is** basically a mobile-first scheduling **and** shift management system.

The first user **is** probably the restaurant manager **or** owner. They need to create and manage shifts.

Actually, we probably need both. Some shifts are assigned directly because they're needed.

For staff, they should be able to log **in and** see "My Shifts" **for** the week. View and accept/reject shifts.

Mobile-first **is** very important. I don't think staff will use desktop. Management tools can be desktop.

The user size per restaurant **is** probably around 50 to 200 staff, but smaller restaurants exist.

Roles are also important. You can't just let anyone claim any shift. A kitchen helper has different shifts than a server.

Availability **is** another thing. Staff should maybe be able to submit availability preferences.

The biggest pain point **is** shift swaps. Right now someone says "Can anyone cover this shift?"

Actually overtime **and** labor rules might matter later, but maybe **not** now. Pay and scheduling rules.

For reminders, we need automatic notifications. Staff should get reminded before their shift.

Managers need a view of coverage. Like **for** each day, are all shifts filled? Can we see gaps?

The scheduling UI should probably have a weekly calendar view. But **for** mobile, a list view might be better.

Authentication-wise, we need manager accounts **and** staff accounts. Staff onboarding.

We should support permissions. Owner/admin can manage billing, locations, staff.

Payroll integration **is** later, but we should know what it means. Could integrate with Stripe.

Important edge cases: staff claiming two overlapping shifts should **not** be allowed.

Maybe there's also a need **for** recurring shifts. Many restaurants have repeat shifts.

Another thing **is** staff availability **and** leave. If someone **is** on leave, they should be marked as unavailable.

I'm **not** sure about pricing but eventually this probably charges per restaurant per month.

The main product objects I imagine are restaurants, locations, users, roles, permissions.

The tone of the product should be very operational **and** practical. Restaurant management.

We may eventually add AI suggestions, like "based on sales forecast, you need more staff on weekends."

For the staff experience, it should be dead simple. They open it **and** see: need to work shifts.

For the manager, the most important screens are probably:

1. weekly schedule,
2. open shift coverage,
3. swap approvals,
4. staff list,
5. notification/reminder settings.

I'm also thinking about audit logs because disputes happen. Like someone say "I didn't work that shift."

We need statuses. Shift could be draft, published, open, filled, completed, closed.

One thing I worry about **is** managers making schedule changes after publishing.

Also, we need to think about bulk actions. Managers don't want to create one shift at a time.

For MVP, maybe the simplest flow **is**:

Manager creates restaurant **and** location.

Manager adds staff **with** names, phone numbers, roles.

Manager creates shifts **for** the week, either assigned **or** open.

Manager publishes schedule.

Staff receive invite/reminder **and** can see shifts.

Staff confirm shifts.

Staff request swaps **or** claim open shifts.

Manager approves swaps.
System sends reminders before shifts.

That's probably the core.

We don't need payroll **in** MVP, but maybe we include export scheduled hours. S

Technically, the API should be built **in** a way that supports mobile clients.

Also, staff count of **50-200** means it's **not** huge scale, but we should support

I don't know how much detail we need **in** the prompt output, but the chain sho

- * product summary,
- * user roles,
- * core entities,
- * workflows,
- * API endpoints,
- * MVP scope,
- * future scope,
- * constraints,
- * edge cases,
- * notification requirements,
- * data model,
- * permissions.

And it should separate what **is** confirmed **from** what **is** optional **or** future. Fo

The biggest thing **is**: don't overbuild. The founder probably wants everything

Also please note that this **is for** small restaurant owners, **not** enterprise. S

CONTEXT:

DOMAIN: Restaurant staff scheduling **and** shift management

PRIMARY USERS:

- Restaurant managers **or** owners (primary user)
- Restaurant staff (servers, kitchen assistants, etc.)

CORE ACTIONS:

1. Managers create **and** publish weekly schedules **with** shifts.
2. Staff view their shifts, claim open shifts, **and** request shift swaps.
3. Managers approve shift swaps **and** manage shift assignments.
4. Staff receive reminders **for** upcoming shifts **and** confirm their shifts.
5. Managers monitor shift coverage **and** address open shifts **or** pending reques

KEY CONSTRAINTS:

- Must be mobile-first; desktop usage **is** secondary.
- Simple **and** lightweight interface; avoid complex HR software features.
- Integration **with** WhatsApp **is** a future consideration, **not** part of MVP.
- Support **for** multiple restaurant locations **and** staff roles.
- Must handle staff availability **and** eligibility **for** shifts.
- Notifications **and** reminders need to be effective **and** timely.

WHAT IS BEING REPLACED:

The current chaotic scheduling process managed through WhatsApp groups, Excel

Identify:

SCALE RISKS: (what breaks when this gets 10x the users)

SECURITY CONCERNS: (auth, data privacy, anything sensitive)

DEPLOYMENT CONSIDERATIONS: (what does this need to run **in** production)

WHAT TO DEFER: (what sounds important but **is not** needed **for** MVP)

Stage 5 — Open Questions

A junior engineer **is** about to start building this.

BRIEF:

So basically the problem we're trying to solve **is**, like, restaurant staff

The current process **is** honestly kind of broken. A manager will usually make

And **for** restaurant owners, this **is** super painful because one missing person

So what we want to build **is** basically a mobile-first scheduling **and** shift ma

The first user **is** probably the restaurant manager **or** owner. They need to cre

Actually, we probably need both. Some shifts are assigned directly because t

For staff, they should be able to log **in and** see "My Shifts" **for** the week. V

Mobile-first **is** very important. I don't think staff will use desktop. Manage

The user size per restaurant **is** probably around 50 to 200 staff, but smaller

Roles are also important. You can't just let anyone claim any shift. A kitch

Availability **is** another thing. Staff should maybe be able to submit availabi

The biggest pain point **is** shift swaps. Right now someone says "Can anyone cc

Actually overtime **and** labor rules might matter later, but maybe **not** now. Pay

For reminders, we need automatic notifications. Staff should get reminded be

Managers need a view of coverage. Like **for** each day, are all shifts filled?

The scheduling UI should probably have a weekly calendar view. But **for** mobil

Authentication-wise, we need manager accounts **and** staff accounts. Staff onb

We should support permissions. Owner/admin can manage billing, locations, st

Payroll integration **is** later, but we should know what it means. Could integr

Important edge cases: staff claiming two overlapping shifts should **not** be al

Maybe there's also a need **for** recurring shifts. Many restaurants have repeat

Another thing **is** staff availability **and** leave. If someone **is** on leave, they

I'm **not** sure about pricing but eventually this probably charges per restaura

The main product objects I imagine are restaurants, locations, users, roles,

The tone of the product should be very operational **and** practical. Restaurant

We may eventually add AI suggestions, like "based on sales forecast, you nee

For the staff experience, it should be dead simple. They open it **and** see: ne

For the manager, the most important screens are probably:

1. weekly schedule,
2. open shift coverage,
3. swap approvals,
4. staff list,
5. notification/reminder settings.

I'm also thinking about audit logs because disputes happen. Like someone say

We need statuses. Shift could be draft, published, open, filled, completed,

One thing I worry about **is** managers making schedule changes after publishing

Also, we need to think about bulk actions. Managers don't want to create one

For MVP, maybe the simplest flow **is**:
Manager creates restaurant **and** location.
Manager adds staff **with** names, phone numbers, roles.
Manager creates shifts **for** the week, either assigned **or** open.
Manager publishes schedule.
Staff receive invite/reminder **and** can see shifts.
Staff confirm shifts.
Staff request swaps **or** claim open shifts.
Manager approves swaps.
System sends reminders before shifts.

That's probably the core.

We don't need payroll **in** MVP, but maybe we include export scheduled hours. S

Technically, the API should be built **in** a way that supports mobile clients.

Also, staff count of **50-200** means it's **not** huge scale, but we should support

I don't know how much detail we need **in** the prompt output, but the chain shc

- ★ product summary,
- ★ user roles,
- ★ core entities,
- ★ workflows,
- ★ API endpoints,
- ★ MVP scope,
- ★ future scope,
- ★ constraints,
- ★ edge cases,
- ★ notification requirements,
- ★ data model,
- ★ permissions.

And it should separate what **is** confirmed **from** what **is** optional **or** future. Fo

The biggest thing **is**: don't overbuild. The founder probably wants everything

Also please note that this **is for** small restaurant owners, **not** enterprise. S

CONTEXT:

DOMAIN: Restaurant staff scheduling **and** shift management

PRIMARY USERS:

- Restaurant managers **or** owners (primary user)

- Restaurant staff (servers, kitchen assistants, etc.)

CORE ACTIONS:

1. Managers create **and** publish weekly schedules **with** shifts.
2. Staff view their shifts, claim open shifts, **and** request shift swaps.
3. Managers approve shift swaps **and** manage shift assignments.
4. Staff receive reminders **for** upcoming shifts **and** confirm their shifts.
5. Managers monitor shift coverage **and** address open shifts **or** pending requests.

KEY CONSTRAINTS:

- Must be mobile-first; desktop usage **is** secondary.
- Simple **and** lightweight interface; avoid complex HR software features.
- Integration **with** WhatsApp **is** a future consideration, **not** part of MVP.
- Support **for** multiple restaurant locations **and** staff roles.
- Must handle staff availability **and** eligibility **for** shifts.
- Notifications **and** reminders need to be effective **and** timely.

WHAT IS BEING REPLACED:

The current chaotic scheduling process managed through WhatsApp groups, Excel

PRODUCTION FLAGS:

SCALE RISKS:

1. ****Database Performance****: With an increase **in** users (e.g., 10x), the data
2. ****API Rate Limiting****: Increased API calls **from** multiple users could exce
3. ****Notification Overload****: A surge **in** notifications (e.g., reminders, app
4. ****Real-time Updates****: If implemented, real-time updates may become ineff
5. ****User Interface Clutter****: A larger user base may lead to more complexit

SECURITY CONCERNS:

1. ****Authentication****: Ensure secure authentication methods (e.g., OTP) to p
2. ****Data Privacy****: Protect sensitive user data (e.g., phone numbers, shift
3. ****Role-based Access Control****: Implement strict permissions to ensure use
4. ****Audit Logging****: Maintain logs of all significant actions (e.g., shift
5. ****Data Breaches****: Regularly assess vulnerabilities **and** ensure compliance

DEPLOYMENT CONSIDERATIONS:

1. ****Hosting Environment****: Choose a scalable cloud provider (e.g., AWS, GCP
2. ****Database Management****: Use a relational database (e.g., PostgreSQL) **wit**
3. ****Load Balancing****: Implement load balancers to distribute traffic evenly
4. ****Monitoring and Logging****: Set up monitoring tools (e.g., New Relic, Dat
5. ****Backup and Recovery****: Establish regular backup procedures **and** a disast

WHAT TO DEFER:

1. ****Payroll Integration****: Integrating **with** payroll systems can be complex
2. ****Advanced Availability Features****: Recurring availability **and** detailed a
3. ****WhatsApp Integration****: While valuable, this can be deferred to a later

4. **AI Scheduling Suggestions**: AI-driven recommendations for staffing can
5. **Complex Reporting and Analytics**: Detailed analytics and reporting fea

List the open questions they MUST get answered before writing a single line of code.

For each question:

- The question itself
- Why it blocks development if unanswered

Stage 6 – Task Breakdown

Who should answer it (founder, designer, legal, etc)

Create a week 1 task breakdown for a junior engineer starting this project.

CONTEXT:

DOMAIN: Restaurant staff scheduling and shift management

PRIMARY USERS:

- Restaurant managers or owners (primary user)
- Restaurant staff (servers, kitchen assistants, etc.)

CORE ACTIONS:

1. Managers create and publish weekly schedules with shifts.
2. Staff view their shifts, claim open shifts, and request shift swaps.
3. Managers approve shift swaps and manage shift assignments.
4. Staff receive reminders for upcoming shifts and confirm their shifts.
5. Managers monitor shift coverage and address open shifts or pending requests.

KEY CONSTRAINTS:

- Must be mobile-first; desktop usage is secondary.
- Simple and lightweight interface; avoid complex HR software features.
- Integration with WhatsApp is a future consideration, not part of MVP.
- Support for multiple restaurant locations and staff roles.
- Must handle staff availability and eligibility for shifts.
- Notifications and reminders need to be effective and timely.

WHAT IS BEING REPLACED:

The current chaotic scheduling process managed through WhatsApp groups, Excel

DATA MODEL:

Database Tables for MVP

1. **restaurants**

- **id** (UUID, Primary Key)
Stores the unique identifier for each restaurant.
- **name** (VARCHAR)
Stores the name of the restaurant.
- **created_at** (TIMESTAMP)
Stores the timestamp when the restaurant was created.

- **updated_at** (TIMESTAMP)
Stores the timestamp when the restaurant was last updated.

2. **locations**

- **id** (UUID, Primary Key)
Stores the unique identifier for each location.
- **restaurant_id** (UUID, Foreign Key)
References the restaurant this location belongs to.
- **name** (VARCHAR)
Stores the name of the location (e.g., Outlet A).
- **timezone** (VARCHAR)
Stores the timezone for scheduling shifts at this location.
- **created_at** (TIMESTAMP)
Stores the timestamp when the location was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the location was last updated.

3. **users**

- **id** (UUID, Primary Key)
Stores the unique identifier for each user (staff or manager).
- **restaurant_id** (UUID, Foreign Key)
References the restaurant this user belongs to.
- **name** (VARCHAR)
Stores the name of the user.
- **phone_number** (VARCHAR)
Stores the phone number for user authentication and notifications.
- **role** (VARCHAR)
Stores the role of the user (e.g., server, manager).
- **created_at** (TIMESTAMP)
Stores the timestamp when the user was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the user was last updated.

4. **shifts**

- **id** (UUID, Primary Key)
Stores the unique identifier for each shift.
- **location_id** (UUID, Foreign Key)
References the location where the shift is scheduled.
- **date** (DATE)
Stores the date of the shift.
- **start_time** (TIME)
Stores the start time of the shift.
- **end_time** (TIME)
Stores the end time of the shift.
- **role** (VARCHAR)
Stores the role required for the shift (e.g., server, cook).

- **number_of_people** (INTEGER)
Stores the number of staff needed **for** the shift.
- **status** (VARCHAR)
Stores the status of the shift (e.g., draft, published, open, filled).
- **created_at** (TIMESTAMP)
Stores the timestamp when the shift was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the shift was last updated.

5. **shift_assignments**

- **id** (UUID, Primary Key)
Stores the unique identifier **for** each shift assignment.
- **shift_id** (UUID, Foreign Key)
References the shift being assigned.
- **user_id** (UUID, Foreign Key)
References the user assigned to the shift.
- **status** (VARCHAR)
Stores the status of the assignment (e.g., assigned, confirmed, pending).
- **created_at** (TIMESTAMP)
Stores the timestamp when the assignment was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the assignment was last updated.

6. **swap_requests**

- **id** (UUID, Primary Key)
Stores the unique identifier **for** each swap request.
- **shift_assignment_id** (UUID, Foreign Key)
References the shift assignment being swapped.
- **requesting_user_id** (UUID, Foreign Key)
References the user requesting the swap.
- **target_user_id** (UUID, Foreign Key)
References the user **with** whom the swap **is** requested.
- **status** (VARCHAR)
Stores the status of the swap request (e.g., pending, approved, rejected).
- **created_at** (TIMESTAMP)
Stores the timestamp when the swap request was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the swap request was last updated.

7. **notifications**

- **id** (UUID, Primary Key)
Stores the unique identifier **for** each notification.
- **user_id** (UUID, Foreign Key)
References the user to whom the notification **is** sent.
- **message** (TEXT)
Stores the content of the notification.

- **sent_at** (TIMESTAMP)
Stores the timestamp when the notification was sent.
- **type** (VARCHAR)
Stores the type of notification (e.g., reminder, confirmation, alert).

8. **availability**

- **id** (UUID, Primary Key)
Stores the unique identifier **for** each availability entry.
- **user_id** (UUID, Foreign Key)
References the user indicating their availability.
- **date** (DATE)
Stores the date **for** which the availability **is** indicated.
- **is_available** (BOOLEAN)
Indicates whether the user **is** available on that date.
- **created_at** (TIMESTAMP)
Stores the timestamp when the availability was created.
- **updated_at** (TIMESTAMP)
Stores the timestamp when the availability was last updated.

Summary

This database schema supports the core functionalities required **for** the MVP,

API DESIGN:

REST API Design

Restaurants

1. **POST /restaurants**

- Creates a new restaurant.
- **Request Body**:
``json
{
 "name": "string"
}
...```
- **Response**:
``json
{
 "id": "UUID",
 "name": "string",
 "created_at": "TIMESTAMP",
 "updated_at": "TIMESTAMP"
}
...```

2. **GET /restaurants**

- Retrieves a list of all restaurants.

```
- **Response:**  
``json  
[  
  {  
    "id": "UUID",  
    "name": "string",  
    "created_at": "TIMESTAMP",  
    "updated_at": "TIMESTAMP"  
  }  
]  
...
```

Locations

1. **POST /locations**

- Creates a new location for a restaurant.

- **Request Body:**

```
``json  
{  
  "restaurant_id": "UUID",  
  "name": "string",  
  "timezone": "string"  
}  
...
```

- **Response:**

```
``json  
{  
  "id": "UUID",  
  "restaurant_id": "UUID",  
  "name": "string",  
  "timezone": "string",  
  "created_at": "TIMESTAMP",  
  "updated_at": "TIMESTAMP"  
}  
...
```

2. **GET /locations**

- Retrieves a list of all locations.

- **Response:**

```
``json  
[  
  {  
    "id": "UUID",  
    "restaurant_id": "UUID",  
    "name": "string",  
    "timezone": "string",  
    "created_at": "TIMESTAMP",  
    "updated_at": "TIMESTAMP"  
  }  
]
```

```
        "updated_at": "TIMESTAMP"
      }
    ]
    ...

#### Users
1. **POST /users**
  - Creates a new user for a restaurant.
  - **Request Body:**
    ```json
 {
 "restaurant_id": "UUID",
 "name": "string",
 "phone_number": "string",
 "role": "string"
 }
 ...
 - **Response:**
    ```json
    {
      "id": "UUID",
      "restaurant_id": "UUID",
      "name": "string",
      "phone_number": "string",
      "role": "string",
      "created_at": "TIMESTAMP",
      "updated_at": "TIMESTAMP"
    }
    ...

2. **GET /users**
  - Retrieves a list of all users.
  - **Response:**
    ```json
 [
 {
 "id": "UUID",
 "restaurant_id": "UUID",
 "name": "string",
 "phone_number": "string",
 "role": "string",
 "created_at": "TIMESTAMP",
 "updated_at": "TIMESTAMP"
 }
]
 ...
```

#### #### Shifts

##### 1. \*\*POST /shifts\*\*

- Creates a new shift for a location.

- \*\*Request Body:\*\*

```
```json
{
  "location_id": "UUID",
  "date": "DATE",
  "start_time": "TIME",
  "end_time": "TIME",
  "role": "string",
  "number_of_people": "integer",
  "status": "string"
}
```
```

- \*\*Response:\*\*

```
```json
{
  "id": "UUID",
  "location_id": "UUID",
  "date": "DATE",
  "start_time": "TIME",
  "end_time": "TIME",
  "role": "string",
  "number_of_people": "integer",
  "status": "string",
  "created_at": "TIMESTAMP",
  "updated_at": "TIMESTAMP"
}
```
```

##### 2. \*\*GET /shifts\*\*

- Retrieves a list of all shifts.

- \*\*Response:\*\*

```
```json
[
  {
    "id": "UUID",
    "location_id": "UUID",
    "date": "DATE",
    "start_time": "TIME",
    "end_time": "TIME",
    "role": "string",
    "number_of_people": "integer",
    "status": "string",
  }
]
```

```
        "created_at": "TIMESTAMP",
        "updated_at": "TIMESTAMP"
    }
]
...

#### Shift Assignments

1. **POST /shift_assignments**
    - Assigns a user to a shift.
    - **Request Body:**
        ```json
 {
 "shift_id": "UUID",
 "user_id": "UUID",
 "status": "string"
 }
 ...
 - **Response:**
        ```json
        {
            "id": "UUID",
            "shift_id": "UUID",
            "user_id": "UUID",
            "status": "string",
            "created_at": "TIMESTAMP",
            "updated_at": "TIMESTAMP"
        }
        ...

2. **GET /shift_assignments**
    - Retrieves a list of all shift assignments.
    - **Response:**
        ```json
 [
 {
 "id": "UUID",
 "shift_id": "UUID",
 "user_id": "UUID",
 "status": "string",
 "created_at": "TIMESTAMP",
 "updated_at": "TIMESTAMP"
 }
]
 ...

Swap Requests
```

### 1. **POST** /swap\_requests

- Creates a new swap request for a shift assignment.

#### - **Request Body:**

```
```json
{
  "shift_assignment_id": "UUID",
  "requesting_user_id": "UUID",
  "target_user_id": "UUID",
  "status": "string"
}
```
```

#### - **Response:**

```
```json
{
  "id": "UUID",
  "shift_assignment_id": "UUID",
  "requesting_user_id": "UUID",
  "target_user_id": "UUID",
  "status": "string",
  "created_at": "TIMESTAMP",
  "updated_at": "TIMESTAMP"
}
```
```

### 2. **GET** /swap\_requests

- Retrieves a list of all swap requests.

#### - **Response:**

```
```json
[
  {
    "id": "UUID",
    "shift_assignment_id": "UUID",
    "requesting_user_id": "UUID",
    "target_user_id": "UUID",
    "status": "string",
    "created_at": "TIMESTAMP",
    "updated_at": "TIMESTAMP"
  }
]
```
```

### #### Notifications

#### 1. **POST** /notifications

- Sends a notification to a user.

#### - **Request Body:**

```
```json
```

```

    {
      "user_id": "UUID",
      "message": "string",
      "type": "string"
    }
    ...

- **Response:**
  ```json
 {
 "id": "UUID",
 "user_id": "UUID",
 "message": "string",
 "sent_at": "TIMESTAMP",
 "type": "string"
 }
 ...

```

## 2. \*\*GET /notifications\*\*

```

- Retrieves a list of notifications for all users.
- **Response:**
  ```json
  [
    {
      "id": "UUID",
      "user_id": "UUID",
      "message": "string",
      "sent_at": "TIMESTAMP",
      "type": "string"
    }
  ]
  ...

```

Availability

1. **POST /availability**

```

- Submits availability for a user.
- **Request Body:**
  ```json
 {
 "user_id": "UUID",
 "date": "DATE",
 "is_available": "boolean"
 }
 ...

- **Response:**
  ```json
  {

```



```
"id": "UUID",
"user_id": "UUID",
"date": "DATE",
"is_available": "boolean",
"created_at": "TIMESTAMP",
"updated_at": "TIMESTAMP"
}
...
```

2. **GET /availability**

– Retrieves a list of availability entries **for** all users.

– **Response:**

```
```json
[
 {
 "id": "UUID",
 "user_id": "UUID",
 "date": "DATE",
 "is_available": "boolean",
 "created_at": "TIMESTAMP",
 "updated_at": "TIMESTAMP"
 }
]
...
```

Output:

MUST BUILD (week 1): (3-5 tasks that unlock everything **else**)

DEFER TO WEEK 2: (important but **not** blocking)

DO NOT TOUCH YET: (things that will waste time now)

For each task **in** MUST BUILD, add one sentence on why it comes first.